CB.EN.U4AIE22132 || MITHIN DEV

`Function`

Practice Problems

1. Sort the hyphen seperated string and return it as hyphen seperated string

In [1]:
```
def hyphensep(words):
    words = words.split("-")
    words.sort()
    string = ""
    for x in words:
        string+= f"{x}-"
    return string[:-1]

def hyphensep_join(words):
    words = words.split("-")
    words.sort()
    return "-".join(words)

word = "Hello-This-Is-Mithin-Dev"
print(hyphensep(word))
print(hyphensep_join(word))
```

```
Dev-Hello-Is-Mithin-This
Dev-Hello-Is-Mithin-This
```

1. Generate a list in a range (Containing Squares)

In [2]:
```
def listgen_square(start,end):
    return [x*x for x in range(start,end+1)]

print(listgen_square(2,10))
```

```
[4, 9, 16, 25, 36, 49, 64, 81, 100]
```

1. Remove duplicates from a sorted list

In [3]:
```
def remove_duplicates(ls):
    if len(ls)<=1:
        return ls
    i = 0
    j = 1
    while j <len(ls):
        if ls[i] == ls[j]:
            ls.pop(j)
        else:
            j+=1
            i+=1
    return ls

ls = [1,1,1,2,3,4,5,5,5,6,6,7,7,8]
print(remove_duplicates(ls))
```

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

1. Change the position of the vowels

In [4]:
```
def change_vowels(string):
    vow=[]
    ind=[]
    for x in range(len(string)):
        if string[x] in "aeiouAEIOU":
            vow.append(string[x])
            ind.append(x)
    ind = ind[::-1]
    ls = list(string)
    for x in range(len(ind)):
        ls[ind[x]] = vow[x]
    return "".join(ls)

word = "Hello World"
print(change_vowels(word))
```

```
Hollo Werld
```

`Error Finding`

`Question 1`

In [ ]:
```
def concatenate_strings(str1, str2):
      return str1 + " " + str2
result = concatenate_strings("Test", 3)
print(result)

# TypeError : Both the arguments should be a string in order to perform concatination

"""
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[3], line 3
      1 def concatenate_strings(str1, str2):
      2     return str1 + " " + str2
----> 3 result = concatenate_strings("Test", 3)
      4 print(result)
      6 # Both the arguments should be a string in order to perform concatination

Cell In[3], line 2, in concatenate_strings(str1, str2)
      1 def concatenate_strings(str1, str2):
----> 2     return str1 + " " + str2

TypeError: can only concatenate str (not "int") to str
"""
```

`Corrected Code`

In [5]:
```
# Corrected Code :

def concatenate_strings(str1, str2):
    return str(str1) + " " + str(str2)
result = concatenate_strings("Test", 3)
print(result)
```

```
Test 3
```

`Question 2`

In [ ]:
```
numbers = {1: "one", 2: "two", 3: "three"}
for number in numbers:
    if numbers[number] == "two":
        del numbers[number]
print(numbers)

# Run The size of the dictionary is changed during the excecution
"""
```

```
---------------------------------------------------------------------------
RuntimeError                              Traceback (most recent call last)
Cell In[9], line 2
      1 numbers = {1: "one", 2: "two", 3: "three"}
----> 2 for number in numbers:
      3     if numbers[number] == "two":
      4         del numbers[number]

RuntimeError: dictionary changed size during iteration
"""
```

Corrected Code

In [10]:
```
numbers = {1: "one", 2: "two", 3: "three"}
for number in list(numbers.keys()):
    if numbers[number] == "two":
        del numbers[number]
print(numbers)
```

```
{1: 'one', 3: 'three'}
```

Practice Problems

```
Given a string and a string dictionary, find the longest string in the
dictionary
that can be formed by deleting some characters of the given string. If
there are
more than one possible results, return the longest word with the smallest
lexicographical order. If there is no possible result, return the empty
string.
Example:
Input: d = {"ale", "apple", "monkey", "plea"}
S = "abpcplea"
Output: "apple"
Explanation: After deleting "b", "c" ,"a" S became "apple" which is
present in d.
```

In [9]:
```
def findLongestWord(s, d):
    d = sorted(d, key=lambda x: (-len(x), x))
    for word in d:
        if isSubsequence(word,s):
            return word
    return ""

def isSubsequence(word,s):
        i, j = 0, 0
        while i < len(s) and j < len(word):
            if s[i] == word[j]:
                j += 1
            i += 1
        return j == len(word)

d = {"ale", "apple", "monkey", "plea"}
S = "abpcplea"
result = findLongestWord(S, d)
print(result)
```

```
apple
```